

Module 10 — First-Run Setup

1. Purpose and Scope

A freshly installed FinManager has an empty database. Nothing in the boot path creates a user: `docker-entrypoint.sh` runs `alembic upgrade head` and nothing else, and the demo dataset is an opt-in command (`./fm seed`) that a real household should never run. Until now that left a clean installation with no way in — the only account came from the seed, with a documented default password.

This module defines the one moment the API accepts an unauthenticated write: the creation of the household and its first **OWNER**. It closes permanently as soon as one user exists.

It **adds no table and no column**. Everything it writes — **Household**, **User**, **HouseholdMember**, **Entity**, **Setting** — is defined by [Module 7](#) and §1a, and is written through the same rules.

Actors

- **The person installing FinManager**, before they are anyone in the system. They have shell access to the host and nothing else.
- Every subsequent member is created by that owner from within the application (M7 FR-7.4). This module never grows a second user.

User Stories

US01. A Way In

As someone who has just run `docker compose up` for the first time, I want the application to ask me to create my account, instead of showing a login form for credentials that do not exist.

US02. No Default Password

As the owner of a self-hosted server, I do not want my installation to ship with a known password that I must remember to change — the window between first boot and that change is the whole vulnerability.

US03. Straight In

As the person who just chose a password, I want to land in the application, not back at a login form retyping what I typed ten seconds ago.

US04. The Door Closes

As the owner of a running installation, I want the setup route to be inert. Nobody who reaches my server should ever be able to create a second owner through it.

US05. Demo Data Stays Optional

As a real household, I want my first login to show an empty, honest application — the Portuguese demo dataset is a developer convenience, not part of installation.

Data Model

No schema change. No table is added, altered or dropped, and this module requires no migration.

What one successful setup writes, exactly once:

Row	Value
Household	name from the form
User	role = OWNER, is_dependent = false, Argon2 password hash, must_change_password = false
HouseholdMember	that user, role = OWNER
Entity	single-member entity named after the owner (M7 FR-7.7)
Setting	every key in the application defaults
AuditLog	one CREATE on users, actor = the new owner themselves

must_change_password is false because the person chose this password seconds ago; there is nothing to rotate. This is the one account that never receives a temporary password.

Integration Contract

Exposes

- **GET /setup/status** → { needs_setup: boolean }. Unauthenticated, safe, cacheable per boot. Reports whether the installation has any non-deleted user.
- **POST /setup** → SessionOut, 201. Unauthenticated. Creates the rows above and returns an authenticated session (cookie + CSRF token), identical in shape to **POST /auth/login**.

Consumes — Household, User, HouseholdMember, Entity, Setting, AuditLog (§1a / M7), the session and CSRF machinery (M7 FR-7.8), and the password hasher.

Functional Requirements

- **FR-7.9 First-Run Bootstrap.** While the installation has no non-deleted User, **POST /setup** creates the household, its first OWNER, that owner's membership, their single-member entity and the default settings, then signs them in. As soon as one user exists, the endpoint returns 409 and **GET /setup/status** reports needs_setup: false. The guard is re-evaluated inside the writing transaction — the status probe is advisory only.
- **FR-7.10 No Seeded Credentials.** The application ships with no default account. **BOOTSTRAP_OWNER_*** remains what it always was: input to the opt-in demo seed, never to a production install.
- **FR-7.11 Setup Password Floor.** The first owner's password is at least 12 characters — longer than the 8 required of invited members, because it cannot be reset from inside the application by anyone else.
- **FR-7.12 Rate Limit.** **POST /setup** is rate-limited per client like login, on the tightest budget in the table, because it is the only unauthenticated write in the API.

UI / Screens

UX-10.1 Setup Screen

Reached automatically: when there is no session, the SPA asks `GET /setup/status` and renders the setup screen instead of the login screen when `needs_setup` is true. A configured installation never pays for the probe beyond one request per unauthenticated boot.

The form asks for four things and nothing else: household name (pre-filled «Casa»), the owner's display name, email, and a password typed twice. It says plainly that there is no email-based recovery.

UX-10.2 Login Screen

Unchanged. It is simply no longer the first thing a clean installation sees.

Analytics & KPIs

None. This module produces no numbers.

Edge Cases & Business Rules

- **Two people racing the form.** The `needs_setup` check runs again inside the transaction, so the second request loses with a `409` rather than creating a second owner.
- **A half-configured database.** A `Household` row with no users (possible only by hand) is reused rather than duplicated; setup is about the missing owner.
- **The demo seed afterwards.** `./fm seed` remains idempotent and additive: it reuses the existing household and adds its own users and data. Running it on a configured install is a developer action, not an installation step.
- **A deleted last owner.** M7 already refuses to remove the last `OWNER`, so an installation cannot fall back into the setup state and reopen the endpoint.
- **CSRF.** `POST /setup` has no session to double-submit against, so it carries no CSRF token — it is unauthenticated by definition and defended by the one-shot guard and the rate limit instead.

Deferred (explicitly not built now)

- Multi-household installations, or a setup flow that creates more than one user.
- Email verification, password-reset-by-email, or SMTP configuration of any kind.
- Importing an existing installation's data during setup.
- A setup wizard covering settings (Brickset key, thresholds); those already have a Settings screen and sensible defaults.
- Re-opening setup through an environment flag — a documented way to create an owner on a live database is a backdoor by another name.

Definition of Done

- ☐ No migration: `alembic heads` is unchanged by this module.
- ☐ `docker compose up` on an empty volume, with no seed, reaches a usable application through the browser alone.
- ☐ `GET /setup/status` reports `true` on an empty database and `false` forever after the first owner exists.
- ☐ `POST /setup` returns `201` with a working session cookie once, and `409` every time after.

- ☐ The created owner is **OWNER**, has an Argon2 hash, **must_change_password** is **false**, and has a single-member entity.
- ☐ A password shorter than 12 characters is refused with **422**.
- ☐ The setup route is in the rate-limit table.
- ☐ Integration tests cover: clean-install probe, successful bootstrap, the door closing, and the password floor.

Open Questions / Decisions

- **Why not keep the seeded owner?** See [ADR-0011](#).